by the rendering engine. So, we will look at a rendering engine—OGRE 3D to be specific.

A very integral part of your game experience is also the sound design and implementation. Without good sound integration, it will be very difficult for a game to sweep you off your feet. We will look at one of the cutting-edge sound libraries out there, called Open AL.

Now, since a game is an emulation of the real world, it has to have the richness and detail of the real world. There are two aspects to this: visual richness and behavioural richness. Things not only need to look real, they also have to behave real. The models and textures used in your game, plus the capabilities of your rendering engine will determine the first aspect, while the physics and AI employed in your game determine the later. We will look at Blender and how it can help you with visual richness, while we will consider the Bullet Physics library to see how physics and AI can be taken care of.

Lastly, a great multi-player experience is something that adds a lot of value to your game in the current networked world. We will look at how RakNet helps in adding that bang to your multi-player experience.

Since we are looking at five tools—all in one—we will keep it simple this time and work individually with each tool at a later opportunity. Also, since we don't claim to be experts in the field and are only learning, most of the content here is derived from our researches over the Internet, rather than personal experience.

OGRE 3D

Object-Oriented Graphics Rendering Engine (or OGRE, for short) is a highly modular graphics rendering engine. Its job is to interface through the available render systems to connect to the underlying graphics hardware on your computer, process the required data and send it to the display device through the graphics hardware. To put it more clearly: OGRE takes all your levels, players, along with the pretty images and sends it to your graphics card for processing. It does so by connecting to one of the two interfaces available -- DirectX or OpenGL. Through one of these render systems, the data is sent to the graphics card to be processed and displayed on your screen.

OGRE is written in objected-oriented C++ with many language bindings available for developers using other languages. It is clean and easily-understandable code, has an immense amount of documentation (API, wikis and forums) and a highly active developer community (forums and IRC channels), which makes it a suitable choice for all sorts of development requirements. It sure does take an experienced C++ developer with some graphics programming experience to actually go into the details and understand the inner working of the behemoth, but with a good knowledge of STL and object-oriented programming in C++, you can be well on your way to making games or other applications that require a 3D rendering engine.

OGRE is highly modular—its pluggable architecture makes adding new features a cakewalk. From custom scene graphs—that is, compositors that post process your screens to input and GUI systems—almost everything can be plugged in and used.This level of abstraction from the underlying system gives developers more time to work on the bigger picture rather than get down and dirty, pushing vertices around.

OGRE is a scene graph-based engine, with all of OGRE's scene managers plugged in to the root of the engine. OGRE comes with an octree, BSP and Paging Landscape scene manager that has support for progressive LOD (level of detail)—automatically or manually created—with many others written and supported by its 10-year-old community.

This multi-platform engine runs on Linux, Mac, Windows and other platforms like the XBox360, PS3 and the iPhone, to name a few. Its flexible licence makes it an optimal choice for both Free/OSS and commercial projects. OGRE was earlier distributed under the GPL and LGPL, but from 1.7 (Cthugha) onwards, it uses the MIT License.

OGRE is purely hardware driven—it runs on your graphics hardware, dedicated or otherwise. There has always been a question of why unlike other engines (like IRRLICHT, with multiple software renderers for fallback) OGRE sticks to being purely hardware driven. The simple answer is that it focuses on cutting-edge technology and supporting the latest hardware-driven features. OGRE tries to stay lean and mean, focusing on what it does best, and letting others do what they do best. It supports Vertex and Fragment programs as well as shaders written in GLSL, HLSL, Cg and assembler.

A recent addition to OGRE's feature list is the new compositing manager and its scripting language. The compositing manager gives the developers full screen post processing for effects such as blooming, blurring, noise, HDR and much more. OGRE has an animation engine with full hardware skinning support and also complete pose mixing. This enables you to blend between animation poses, giving you smoother and more realistic animation. It also comes with a highly extensible particle manager with customisable emitters and effectors.

Apart from being a wonderful graphics rendering engine, OGRE also deals with minimal windowing, memory debugging, input, GUI systems, resource loading and management, to name a few. There are many community projects that also deal with content creation, as well as the export and import of pipelines for the engine.

OGRE can be seen in action in many free and open source interactive applications and games, as well as in many commercially successful projects like *Jack Keane*, *Venetica*, *Torchlight* and *Zombie Driver*, to name a few.

Open Audio Library (Open AL)

Open AL is a free, cross-platform audio API. The API and